2. Theoretical Application

As luminance is the most important aspect of the image, this essay will primarily focus on grayscale pictures. This will simplify many calculations while providing adequate material to work with. Since (following the sub-sampling step) the treatment of chroma and lumina layers differ only by quantization table, all strategies discussed can be applied to colored images in a similar manner. Thus, color-space conversion and sub-sampling will not be relevant in this paper.

When looking at aspects of the JPEG algorithm to be modified, it becomes clear that not much can be changed for some steps, as these processes are already heavily optimized and leave no room for additional customisation. One of these steps is DCT, which uses a very precise measurement of spatial frequency data to generate its coefficients and cannot be modified. Another such process is encoding, which has already been heavily developed and cannot be further optimized; meaning that all modifications must be in the quantization step.

2.1 The Quantization Table

Common JPEG algorithms use a universal set of values for the quantization table, which promotes compatibility and speed, but lacks individual input. However, this default configuration can be modified to accommodate specific user requirements, allowing for flexibility in the final file size. A new quantization table can be constructed by taking the existing JPEG quantization table and multiplying each value by a constant, k.

Thus, the average length—or the expected value—is the weighted sum:

$$\sum_{i=1}^{16} L_i \times N_i \times 2^{-i} \approx 3.295 \quad (1)$$

The value is rounded to 3 decimal places as any more precision is unnecessary.

2.2.2 Number of Non-Zero Coefficients

Following quantization, the quotient is rounded to the nearest integer, leaving many null coefficients. This means that the threshold for the "zero" coefficients is $|[A_i]| < 0.5$. This parameter can be modified to change the number of non-zero coefficients, which allows the user to change the final size of the image to their liking. Representing this value as T, all non-zero coefficients can now be expressed as $|[A_i]| \geq T$.

Next, the number of non-zero coefficients can be determined. Although this can be accomplished via a program that individually counts the number of non-zero coefficients, a probability distribution curve could also be employed. Using known data from the image pixels, a distribution representing the to-be-stored values can be constructed and used to determine the probability a value is within a certain range. Thus, the multiplying the probability that a coefficient lies outside the zero threshold and the total number of pixels will yield the number of non-zero coefficients:

$$\#\ of\ non-zero\ coefficients\ =\ n \times P(|[A_i]| \geq T)$$

Where n is the number of pixels

As the coefficients are centred and sharply peaked around zero before (and after) quantization, a distribution like the Laplacian best matches the data (Eggerton & Srinath, 2003).

Given a Laplacian distribution in the form:

$$P(X) = \frac{b}{2} e^{-b|x-\mu|}$$

Since DCT coefficients are, by definition, centered around zero, It can be assumed that their mean is also zero:

$$\mu = 0$$

For the following calculations, $[A_i]$ will be expressed as "x" for simplicity and ease of communication.

Knowing the number of non-zero coefficients:

$$n \times P(|[A_i]| \geq T)$$

$$= n \times (1 - P(-T < x < T))$$

$$= n(1 - (\int_{-T}^{0} \frac{b}{2} e^{bx} dx + \int_{0}^{T} \frac{b}{2} e^{-bx} dx))$$

$$= n(1 - \frac{b}{2}(\frac{1}{b} - \frac{1}{b} e^{-T} - \frac{1}{b} e^{-T} + \frac{1}{b}))$$

$$= n(1 - \frac{b}{2}(\frac{2}{b} - \frac{2}{b} e^{-Tb}))$$

$$= ne^{-Tb} \quad (2)$$

Thus, using equation (1), the size of the image after compression can be expressed as:

$$3.295 \times ne^{-Tb}$$

However, before this can be useful, a method of calculating the value of the constant, b, must be determined.

From the definition of variance for a continuous distribution (*Variance and Standard Deviation*, n.d.):

$$\sigma^2 = \int_{-\infty}^{\infty} P(X)(x-\mu)^2 dx$$

$$= \int_{-\infty}^{\infty} x^2 \frac{b}{2} e^{-b|x|}\, dx$$

$$= \frac{b}{2}\int_{-\infty}^{0} x^2 e^{bx}\, dx + \frac{b}{2}\int_{0}^{\infty} x^2 e^{-bx}\, dx$$

$$= \frac{b}{2}\int_{-\infty}^{0} x^2 \left(\frac{e^{bx}}{b}\right)' dx + \frac{b}{2}\int_{0}^{\infty} x^2 \left(-\frac{e^{-bx}}{b}\right)' dx$$

$$= \frac{b}{2}\left(\left[\frac{x^2 e^{bx}}{b}\right]_{-\infty}^{0} - \frac{2}{b}\int_{-\infty}^{0} x e^{bx}\, dx\right) + \frac{b}{2}\left(\left[-\frac{x^2 e^{-bx}}{b}\right]_{0}^{\infty} - \frac{2}{b}\int_{0}^{\infty} -x e^{-bx}\, dx\right)$$

$$= \frac{b}{2}\left(\left[\frac{x^2 e^{bx}}{b}\right]_{-\infty}^{0} - \frac{2}{b}\int_{-\infty}^{0} x\left(\frac{e^{bx}}{b}\right)'\mathrm{dx}\right) + \frac{b}{2}\left(\left[-\frac{x^2 e^{-bx}}{b}\right]_{0}^{\infty} - \frac{2}{b}\int_{0}^{\infty} -x\left(\frac{e^{-bx}}{b}\right)' dx\right)$$

$$= \frac{b}{2}\left(\left[\frac{x^2 e^{bx}}{b}\right]_{-\infty}^{0} - \left[\frac{2x e^{bx}}{b^2}\right]_{-\infty}^{0} + \frac{2}{b^2}\int_{-\infty}^{0} e^{bx}\mathrm{dx}\right) + \frac{b}{2}\left(\left[-\frac{x^2 e^{-bx}}{b}\right]_{0}^{\infty} - \left[\frac{2x e^{-bx}}{b^2}\right]_{0}^{\infty} - \frac{2}{b^2}\int_{0}^{\infty} e^{-bx}\, dx\right)$$

$$= \frac{b}{2}\left[\frac{x^2 e^{bx}}{b} - \frac{2x e^{bx}}{b^2} + \frac{2}{b^3} e^{bx}\right]_{-\infty}^{0} + \frac{b}{2}\left[-\frac{x^2 e^{-bx}}{b} - \frac{2x e^{-bx}}{b^2} - \frac{2}{b^3} e^{-bx}\right]_{0}^{\infty}$$

$$= \frac{b}{2} \times \frac{2}{b^3} + \frac{b}{2} \times \frac{2}{b^3}$$

$$\sigma^2 = \frac{2}{b^2}$$

$$b = \frac{\sqrt{2}}{\sigma}$$

Where σ is the standard deviation of the quantized coefficients.

This relationship between b and the standard deviation serves to calculate b after finding σ from the image.

2.3 Quality Loss

Next, a loss function must be defined, representing the quality loss in the image. This can be defined as the difference between the new image and the original. To do this, the mean squared error can be used, which is often used to determine the accuracy of estimations (Stewart, 2024).

Thus, the loss function would be defined as:

$$\frac{1}{n}\sum_{i=1}^{n}([I_i] - [A_i])^2$$

$$= \frac{1}{n}\sum_{i=1}^{n}\left([I_i] - \frac{[I_i]}{k[Q_i]}\right)^2$$

However, since some coefficients become zero after quantization, only non-zero coefficients should be accounted for. Thus, the quantized value, $[A_i]$, should be multiplied by the probability that it is non-zero, which was determined in equation (2).

$$= \frac{1}{n}\sum_{i=1}^{n}\left([I_i] - \frac{[I_i]}{k[Q_i]} \times e^{-T}\right)^2$$

2.4 Optimization

Finally, the constrained optimization problem can be set up using a Lagrangian, which takes a function to be optimized, and a function to be constrained; it finds the local extremum given that the constrained function is met (Corral, 2023).

To optimize the values of *T* and *k* that yield the minimum amount of data loss while ensuring the desired file size:

$$L(k,T,\lambda) = \frac{1}{n}\sum_{i=1}^{n}\left([I_i] - \frac{[I_i]}{k[Q_i]} \times e^{-Tb}\right)^2 - \lambda(3.295 \times ne^{-Tb} - d)$$

Where d is the desired file size in bits.

This can be solved by splitting into 3 equations:

$$\frac{\partial L}{\partial k} = 0$$

$$\frac{\partial L}{\partial T} = 0$$

$$\frac{\partial L}{\partial \lambda} = 0$$

These equations may be difficult to compute by hand and can instead be solved by a program written in MATLAB.

The code can be found in Appendix C, which uses k=1 and T=0.5, as these would be the default values for the compression of a typical JPEG image. However, the program took too long to run, and eventually timed out.

Recognizing that this error was likely due to computational strain of performing operations with functions involving the sum of thousands, if not millions, of terms that include the symbolic value of k. This burden can be eased by mathematically manipulating the functions to be simpler to compute:

$$b = \sqrt{2} \div \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\frac{[I_i] \times e^{-T}}{k[Q_i]}\right)^2}$$

$$= \sqrt{2}ke^{-Tb} \div \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\frac{[I_i]}{[Q_i]}\right)^2}$$

$$= \frac{\sqrt{2}ke^{-Tb}}{\sigma}$$

Noting that σ is the standard deviation of the list defined as $[IQ_i] = \frac{[I_i]}{[Q_i]}$.

When the new code (Appendix D) was run, it produced output that can be seen in figure 6. This can be compared to the "actual" size of the image which can be seen in figure 7.

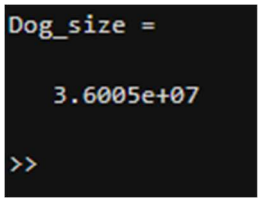


Figure 6: Estimated Image size, original image from MATLAB

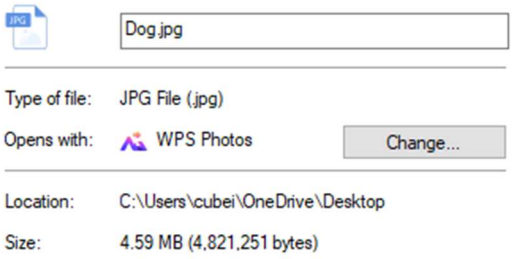


Figure 7: Actual Image Size, original image

Noting that 1 byte is equal to 8 bits, the error can be calculated as the percentage difference between the actual and estimated size of the image (*Bits and Bytes*, n.d.).

Thus, the error in calculation is roughly:

$$\frac{4821251 \times 8 - 3.6005 \times 10^7}{4821251 \times 8} = 6.65\%$$

This process was repeated for 4 unique images to test the viability of the estimation, which resulted in an average error of 5.59% (See Appendix E).

This highlights the system's imperfections as a measurement system; the estimation is slightly above a reasonable range of error of 5% (*Accuracy and Precision*, 2024); the reasonability is reinforced particularly if the algorithm is employed for personal use.

3.2 Testing Optimization Code

After affirming the assumptions, the optimization code can finally be written. Using the DCT coefficients obtained from the dct2() function, the Lagrangian can be set up in the following manner:

Figure 10 shows the output; as each of the subsequent values have a greater magnitude than ‘loss’, it can be concluded that the code does indeed optimize for minimum quality loss.

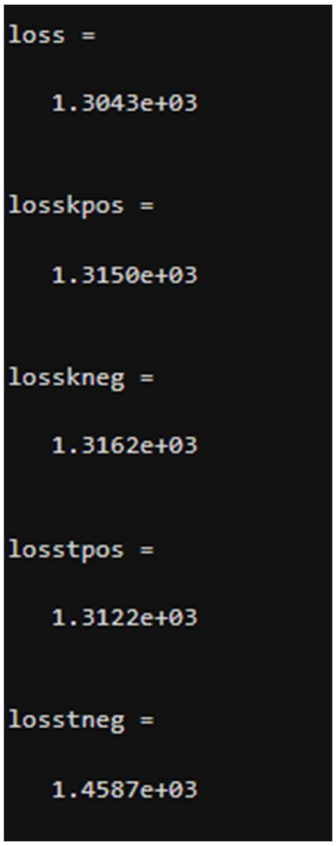


Figure 10: Minimum Point Test, original image from MATLAB

3.3 Optimizing Image

Now the code can be implemented to optimize the compression of real images, as before: where the 1-dimensional arrays I and Q were obtained by storing the DCT coefficients and concatenating the default quantization table, respectively. Unfortunately, this program also ended up timing out; a similar process can be applied to the loss function:

$$f = \frac{1}{n}\sum_{i=1}^{n}\left([I_i] - \frac{[I_i]}{k[Q_i]} \times e^{-Tb}\right)^2$$

$$= \frac{1}{n}\left(\sum_{i=1}^{n}([I_i]^2) - 2\sum_{i=1}^{n}\left(\frac{[I_i]^2 \times e^{-Tb}}{k[Q_i]}\right) + \sum_{i=1}^{n}\left(\frac{[I_i] \times e^{-Tb}}{k[Q_i]}\right)^2\right)$$

$$f = \frac{1}{n}\left(\sum_{i=1}^{n}([I_i]^2) - \frac{2e^{-Tb}}{k}\sum_{i=1}^{n}\left(\frac{[I_i]^2}{[Q_i]}\right) + \frac{e^{-2Tb}}{k^2}\sum_{i=1}^{n}\left(\frac{[I_i]}{[Q_i]}\right)^2\right)$$

This way, the symbolic form of *k* is only accounted for after number crunching within the summation, which should greatly improve the efficiency of the code. The new code can be seen Appendix H. The final output can be seen in Figure 11 below.

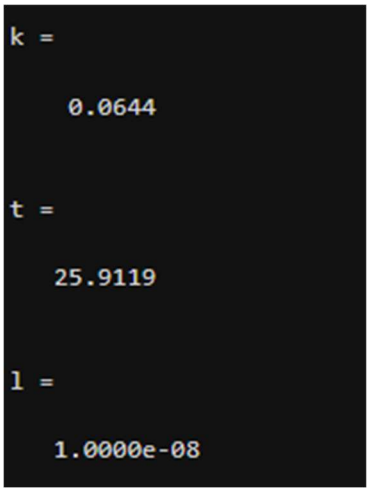


Figure 11: Solutions for "Dog" Image, original image from MATLAB

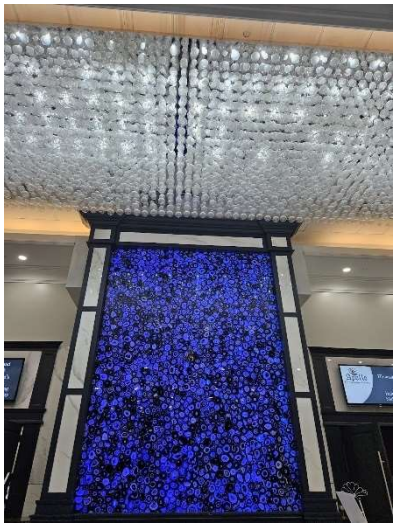
4. Conclusion

4.1 Limitations and Improvements

Despite the relative success, there are still limitations that must be addressed when determining the viability of the results. Many estimates and assumptions were made for simplicity, which directly impacts the accuracy of the results. Firstly, although the "average bits per character" was determined by using a source trained on a large data set, the estimation using probability in a Huffman tree is by no means the most accurate value. Additionally, it is assumed that the coefficients roughly follow a Laplacian distribution, which is supported by researchers Eggerton and Srinath. However, it is important to note that any calculations based on probability will always be an estimation, and must therefore be taken with a grain of salt. Moreover, while traditional JPEG uses a predetermined quantization table and zero-coefficient tolerance, this method calculates unique values for each image, meaning that this information must also be

Appendix E

Other 4 images tested

Image	Estimated Size	Real Size	Percent Error
	3.6105107 bits	37910267 bits	4.76%
	3.6199107 bits	34253278 bits	5.68%

	3.6591107 bits	33160495 bits	10.35%
	3.4970107 bits	35142282 bits	0.49%